

SQL Injection Detection - SOC Incident Analysis

1. Incident Overview

This report documents the investigation of **EventID 115**, which triggered the SOC detection rule **SOC165 - Possible SQL Injection Payload Detected**.

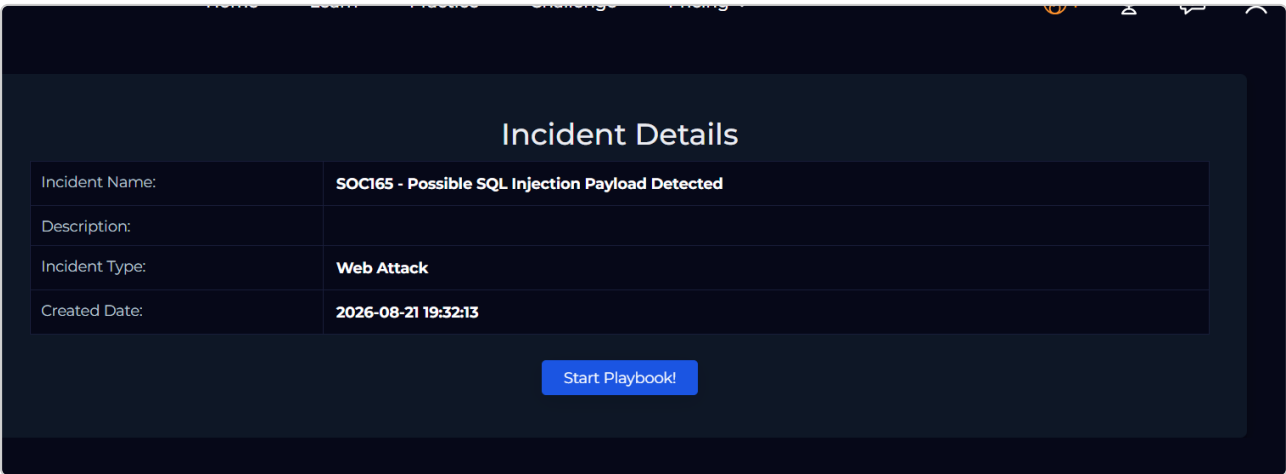
The alert identified a suspicious HTTP GET request directed at the web server **WebServer1001**. The request originated from the external IP address **167.99.169.17** and contained an SQL injection-style payload in the URL query parameter.

The payload included:

```
" OR 1 = 1 -- -
```

The presence of **OR 1 = 1** is a well-known indicator of SQL injection attempts because it creates a condition that evaluates as true. The **--** sequence can be used to comment out the remainder of an SQL statement.

The request was **Allowed** by the security device. However, an allowed request does not confirm that exploitation was successful. Additional application, web-server, and database logs would be required to determine whether the payload was executed successfully.



2. Incident Details

Field	Details
Event ID	115
Event Time	2022-02-25 11:34:32 (+03:00)
Detection Rule	SOC165 - Possible SQL Injection Payload Detected
Role	Security Analyst
Alert Type	Web Attack
Difficulty	Medium
MITRE ATT&CK	T1190 - Exploit Public-Facing Application
Hostname	WebServer1001
Source IP	167.99.169.17
Destination IP	172.16.17.18
HTTP Method	GET

Field	Details
Device Action	Allowed
User-Agent	Mozilla/5.0 (Windows NT 6.1; WOW64; rv:40.0) Gecko/20100101 Firefox/40.1

SEVERITY	DATE	ROLE NAME	EVENTID	TYPE
High	2022-02-25T11:34:32+03:00	SOC165 - Possible SQL Injection Payload Detected	115	Web Attack
EventID : 115 Event Time : 2022-02-25T11:34:32+03:00 Rule : SOC165 - Possible SQL Injection Payload Detected Role : Security Analyst Alert Type : Web Attack Difficulty : Medium MITRE ATT&CK : T1190 Hostname : WebServer1001 User-Agent : Mozilla/5.0 (Windows NT 6.1; WOW64; rv:40.0) Gecko/20100101 Firefox/40.1 Device Action : Allowed Requested URL : https://172.16.17.18/search/?q=%22%20OR%201%20%3D%201%20--%20- Source IP Address : 167.99.169.17 HTTP Request Method : GET Alert Trigger Reason : Requested URL Contains OR 1 = 1 Destination IP Address : 172.16.17.18				

3. Network Traffic Direction

Computer Network → Computer Network

The source IP 167.99.169.17 sent an HTTP request towards the destination web server 172.16.17.18 .

```

Source
167.99.169.17
|
| HTTP GET
| SQL Injection Payload
v
Destination
172.16.17.18
WebServer1001

```

4. Suspicious Request

Requested URL:

```
https://172.16.17.18/search/?q=%22%20OR%201%20%3D%201%20--%20-
```

Decoded:

```
" OR 1 = 1 -- -
```

The SQL injection payload was supplied through the `q` parameter of the `/search/` endpoint.

URL component breakdown

Encoded	Meaning
/search/	Application search endpoint being targeted
?q=	Search query parameter
%22	"
%20	(space)
%3D	=

Decoded, the request contains the SQL-like expression: " OR 1 = 1 -- -

5. SQL Injection Analysis

SQL injection is a web application attack in which an attacker attempts to manipulate an application's database query by inserting SQL syntax into an input field or parameter.

The significant component of the payload is:

```
OR 1 = 1
```

The expression `1 = 1` is always true. An application that improperly constructs SQL queries could interpret attacker-controlled input as part of the SQL statement rather than as ordinary search data. Conceptually, a vulnerable query might look like:

```
SELECT * FROM users
WHERE username = 'INPUT';
```

If untrusted input is inserted directly into the query, malicious input could alter the intended logic of the statement. The `--` sequence is significant because SQL comment syntax can cause the remainder of a query to be ignored in some database systems.

6. Detection Reason

Rule: SOC165 - Possible SQL Injection Payload Detected

Trigger condition: Requested URL Contains `OR 1 = 1`

This detection logic identified a common SQL injection pattern within the HTTP request, providing a strong indicator that the request was not a normal search query.

7. HTTP Request Method

The request used `GET`, which places request parameters directly in the URL — making the payload directly observable in `/search/?q=...` without needing to inspect a request body.

8. Source and Destination Analysis

Source — `167.99.169.17` Should be investigated for additional requests against the web server, other injection payloads, directory enumeration, authentication attacks, vulnerability scanning, requests against other endpoints, and other security alerts.

Destination — 172.16.17.18 (WebServer1001) Should be investigated for evidence of successful exploitation or application compromise.

Collect Data

Gather some information that can be gathered quickly to get a better understanding of the traffic. These can be summarized as follows.

- Ownership of the IP addresses and devices.
- If the traffic is coming from outside (Internet);
 - Ownership of IP address (Static or Pool Address? Who owns it? Is it web hosting?)
 - Reputation of IP Address (Search in VirusTotal, AbuseIPDB, Cisco Talos)
- If the traffic is coming from company network;
 - Hostname of the device
 - Who owns the device (username)
 - Last user logon time

Next

9. User-Agent Analysis

```
Mozilla/5.0 (Windows NT 6.1; WOW64; rv:40.0) Gecko/20100101 Firefox/40.1
```

Indicates a browser identifying as Firefox on Windows. User-Agent values can be spoofed and should not be treated as reliable evidence of the attacker's actual OS or browser — treated as supporting information only, not attribution.

10. Device Action

Action: **Allowed** — the security control permitted the request to continue.

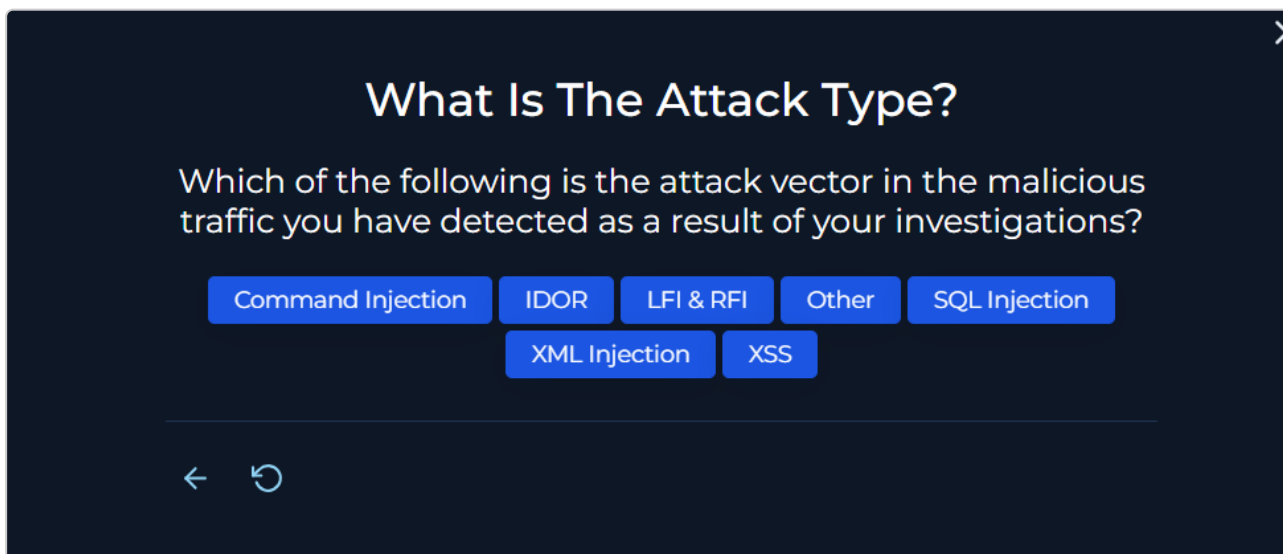
It is important to distinguish between an **attack attempt detected** and an **attack successfully executed**. This event confirms the suspicious request but does not prove the SQL injection succeeded. Further investigation is required to determine whether:

1. The application processed the request.
2. The application generated a database query.
3. The database accepted the query.
4. Sensitive information was returned.
5. Authentication or authorization was bypassed.
6. The attacker obtained access to additional resources.

11. MITRE ATT&CK Mapping

T1190 - Exploit Public-Facing Application

T1190 describes attempts to exploit vulnerabilities in applications exposed to external or untrusted networks. Here, the attacker targeted the web application's search functionality using an SQL injection payload — consistent with the observed activity.



What Is The Attack Type?

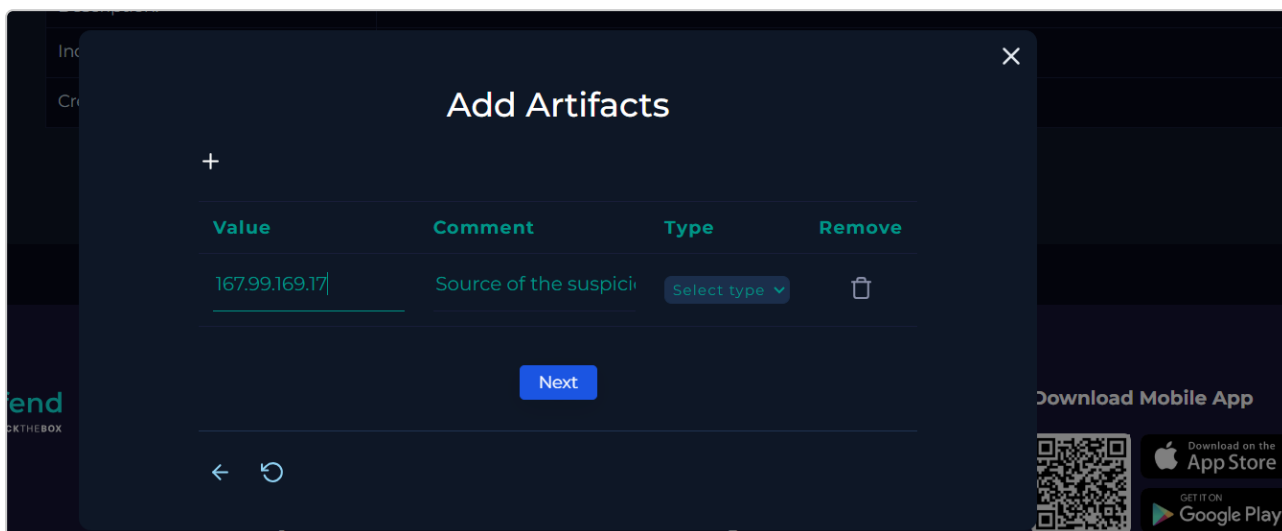
Which of the following is the attack vector in the malicious traffic you have detected as a result of your investigations?

Command Injection IDOR LFI & RFI Other SQL Injection XML Injection XSS

← ↻

12. Indicators of Compromise / Artifacts

Type	Value	Comment
IP Address	167.99.169.17	Source IP associated with the suspected SQL injection request against WebServer1001
URL	https://172.16.17.18/search/?q=%22%20OR%201%20%3D%201%20--%20-	Requested URL contains an SQL injection-style payload using OR 1=1 and SQL comment syntax
Host	WebServer1001	Destination server — should be reviewed for evidence of successful exploitation



Add Artifacts

+

Value	Comment	Type	Remove
167.99.169.17	Source of the suspicious request	Select type ▼	🗑️

Next

← ↻

Download Mobile App

Download on the App Store

GET IT ON Google Play

13. Investigation Findings

1. A suspicious HTTP request was detected against WebServer1001 .
2. The request originated from 167.99.169.17 .
3. The request used the HTTP GET method.

4. The target endpoint was `/search/`.
5. The search parameter contained an SQL injection-style payload.
6. The payload contained `OR 1 = 1`.
7. The payload also contained SQL comment syntax.
8. The SOC rule correctly identified the suspicious pattern.
9. The security device allowed the request.
10. The supplied evidence does not confirm successful exploitation.
11. Additional logs are required to determine the impact.

14. False-Positive Check

Before escalating, planned/authorized testing was ruled out by checking for prior notice of scheduled penetration testing and confirming the source device was not an attack-simulation tool (e.g. Verodin, AttackIQ, Picus).

Check If It Is a Planned Test

Penetration tests or attack simulation products can trigger False Positive alarms if the rules are not set correctly. Check whether the malicious traffic is the result of a planned test.

- Check if there is an email showing that there will be planned work by searching for information such as hostname, username, IP address on the mailbox.
- Check if the device generating malicious traffic belongs to attack simulation products. If the Hostname contains the name of Attack Simulation products (such as Verodin, AttackIQ, Picus...), these devices belong to Attack Simulation products within the framework of LetsDefend simulation and it is a planned work.

Is the malicious traffic caused by a planned test?

Not Planned

Planned

15. True Positive / False Positive Assessment

Classification: TRUE POSITIVE

The request contains a deliberately crafted SQL injection-style expression (`" OR 1 = 1 -- -`), substantially different from normal search input and matching the detection rule's SQL injection criteria.

The classification specifically describes the event as a **detected SQL injection attempt**, rather than claiming the server was successfully compromised — there is currently insufficient evidence to conclude exploitation succeeded.

16. Impact Assessment

Confirmed impact: Suspicious SQL injection request against a web application.

Not yet confirmed: database compromise, data exfiltration, account compromise, authentication bypass, remote code execution, malware installation, privilege escalation.

Impact remains **unconfirmed** until additional evidence is reviewed.

17. Containment

Since the alert warranted a precautionary response, the affected host was reviewed on the Endpoint Security page and contained to restrict any potential attacker activity, prevent spread, and reduce impact while the investigation continued. Containment procedures vary by organization — some grant analysts direct containment authorization, others require escalation.



Containment

Since it is detected that the device is compromised, the device must be isolated in order to restrict the attacker, prevent the spread of the attack and reduce the impact.

Go to the Endpoint Security page and contain the relevant device with the help of the "Request...Containment" button.

- Endpoint Security

** Each institution has a separate containment procedure. While some institutions give containment authorization, some do not. Do not forget to learn the containment procedure in the institution you work for.

Next

18. Recommended Investigation

Web Server Logs — search for `167.99.169.17` and `/search/`; review HTTP status codes, response sizes, repeated requests, error messages, unusual parameters, additional injection payloads.

Application Logs — review activity around `2022-02-25 11:34:32 +03:00` for SQL errors, authentication anomalies, unexpected behaviour, database exceptions, unusual response sizes.

Database Logs — check for unexpected queries, SQL syntax errors, access to sensitive tables, unusual accounts, large data retrieval, queries originating from the web application.

Network Logs — search for further communication involving `167.99.169.17` to identify additional attack activity from the same source.

19. Recommended Security Actions

1. Investigate the source IP across all available security logs.
 2. Review web-server logs around the alert timestamp.
 3. Review application and database logs for evidence of successful SQL injection.
 4. Identify whether the `/search/` endpoint is vulnerable to SQL injection.
 5. Validate that the application uses parameterized queries or prepared statements.
 6. Ensure user input is properly validated and safely handled.
 7. Review whether sensitive database information was exposed.
 8. Monitor the source IP for repeated attack attempts.
 9. Apply appropriate web application security controls.
 10. Consider blocking or rate-limiting the source if malicious activity is confirmed and operationally appropriate.
 11. Review similar alerts to determine whether this was an isolated attempt or part of a wider campaign.
-

20. Detection Improvement

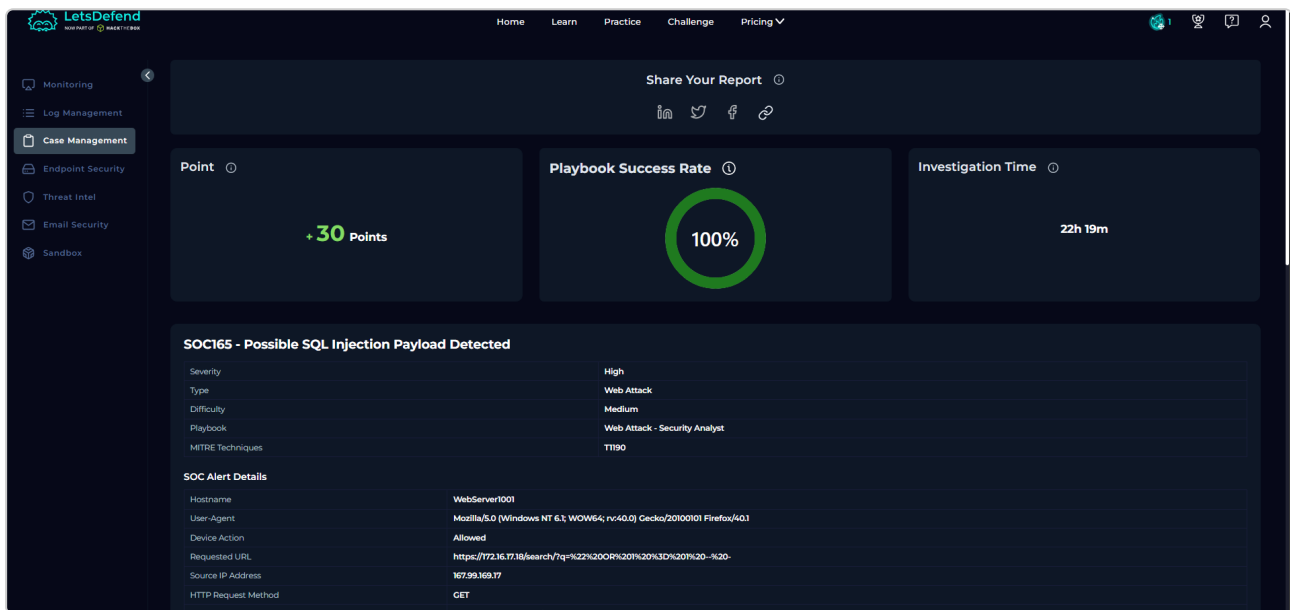
The existing rule successfully detected the payload by matching `OR 1 = 1`. Detection can be improved by monitoring for multiple SQL injection indicators rather than relying on a single string, e.g.:

```
OR 1=1
UNION SELECT
SELECT
DROP TABLE
INSERT INTO
UPDATE
DELETE FROM
--
#
/*
*/
```

Detection should also account for URL encoding, since attackers may encode malicious characters (`%27` , `%22` , `%20` , `%3D`) to bypass simple signature-based rules. Rules should be carefully tuned to reduce false positives caused by legitimate application traffic.

21. Case Closure

The case was closed as a **True Positive** with a 100% playbook accuracy score, completed in 22h 19m investigation time.



22. Final Analyst Assessment

Field	Value
Severity	Medium
Classification	True Positive
Attack Type	SQL Injection
MITRE ATT&CK	T1190 – Exploit Public-Facing Application
Source	167.99.169.17
Destination	172.16.17.18 / WebServer1001
HTTP Method	GET
Action	Allowed
Primary Indicator	OR 1 = 1 -- -
Confirmed Compromise	No evidence from the supplied event
Recommended Priority	Investigate immediately and correlate with web, application, database, and network logs

Summary: The event represents a credible SQL injection attack attempt against a web application. The attacker attempted to manipulate the application's search parameter using a condition that is always true, combined with SQL comment syntax to neutralize the rest of the query. The detection rule correctly identified the suspicious request. Although the request was allowed, successful exploitation cannot be confirmed from this single event, and further log correlation is required to determine whether the application or underlying database was compromised.